

Luminar Per-Point Timestamp Diagnostic Guide

How to run the one-shot [LUMINAR_TS_DIAG] dump in gicp_localization and interpret it to fix point-cloud deskewing.

DLIO++ · art-jazzy branch · package: gicp_localization

1. Background — why this exists

Luminar Iris LiDAR point clouds carry a 64-bit per-point timestamp. Inside gicp_localization we declare the dlio Point structure with an 8-byte timestamp union slot and treat the bytes as a **uint64 in nanoseconds** (see copyPointTimeFromCloud → the SensorType::LUMINAR branch). This matches the team's iris_pcap_reader.cpp, which declares the field UINT8[8] at offset 0 and writes $PTP_sec \times 1e9 + ts_offset_ns$.

When users observe "collapsed per-point timestamps" the most likely explanation is that the *live* Luminar ROS2 driver advertises the timestamp PointField with a **different encoding** than the pcap reader (for example FLOAT64 seconds, or all points stamped to the scan time). If our memcopy reads it as the wrong type, deskewing becomes a no-op or actively wrong.

Rather than guess, the diagnostic dumps the first cloud's bytes, interprets them four ways at once, and lets you pattern-match the actual driver convention in one boot.

2. Where the diagnostic lives

All changes are in gicp_localization/src/localization.cc and require no new launch flag.

```
// Two free functions in the anonymous namespace:
pointFieldDatatypeName(uint8_t datatype) -> const char*
logTimestampDiagnostic(const PointCloud2&, ts_off,
                      ts_datatype, ts_count, sensor_name)

// One std::call_once block inside preprocessPointCloud,
// fired right after findTimeField returns:
static std::once_flag ts_diag_once;
std::call_once(ts_diag_once, [&]() {
    logTimestampDiagnostic(*pc, time_off, time_datatype,
                          time_count, sensor_name);
});
```

It fires **once per node lifetime** (the first cloud after node startup), writes to stderr regardless of localization/verbose, and is always-on. There is no performance impact after the first cloud.

3. How to run the diagnostic

3.1 Build

```
cd ~/colcon_ws
colcon build --packages-select gicp_localization
--cmake-args -DCMAKE_BUILD_TYPE=RelWithDebInfo
source install/setup.bash
```

3.2 Launch with a live Luminar source

Either bring up the car and start the Luminar driver, or replay a bag that contains the raw driver topic (the diagnostic only consumes the first PointCloud2 message, so a 1–2 second clip is enough).

```
ros2 launch gicp_localization localization_with_tf.launch.py
# in another shell, replay or start the lidar driver:
ros2 bag play <bag> --topics /luminar_front_points
--clock --rate 0.5
```

3.3 Capture the dump

The diagnostic block is bracketed by easy-to-grep markers:

```
# Tail node stderr; the block looks like:
# [LUMINAR_TS_DIAG] BEGIN
# ... 20-40 lines ...
# [LUMINAR_TS_DIAG] END

# Quickest capture (works for both ros2 launch and ros2 run):
ros2 launch gicp_localization localization_with_tf.launch.py
2>&1 | tee /tmp/gicp.log
grep --color=never LUMINAR_TS_DIAG /tmp/gicp.log
```

If you do not see the block at all, either the node never received a cloud (check the topic remap and the bag) or another logger is redirecting stderr.

4. Anatomy of the dump

Below is a synthetic example for a Luminar driver that advertises UINT8[8] at offset 0 with proper PTP nanoseconds. Yours will differ in the bytes but the structure is identical.

```
[LUMINAR_TS_DIAG] BEGIN
[LUMINAR_TS_DIAG] sensor_type=luminar point_step=32 num_points=98304
[LUMINAR_TS_DIAG] width=98304 height=1 is_bigendian=0
[LUMINAR_TS_DIAG] header.stamp=1748555700.123456789
[LUMINAR_TS_DIAG] frame_id='luminar_front'
[LUMINAR_TS_DIAG] fields (offset, datatype, count, name):
[LUMINAR_TS_DIAG]   off=0      type=UINT8    count=8    name='t'
[LUMINAR_TS_DIAG]   off=8      type=FLOAT32   count=1    name='x'
[LUMINAR_TS_DIAG]   off=12     type=FLOAT32   count=1    name='y'
[LUMINAR_TS_DIAG]   off=16     type=FLOAT32   count=1    name='z'
[LUMINAR_TS_DIAG]   off=20     type=FLOAT32   count=1    name='intensity'
[LUMINAR_TS_DIAG] timestamp_field: off=0 type=UINT8 count=8 implied_byte_size=8
[LUMINAR_TS_DIAG] point[0]      idx=0      raw=15 84 1e cd dd a4 47 18
[LUMINAR_TS_DIAG]                  as_uint64=1748555700123456789
[LUMINAR_TS_DIAG]                  as_double_sec=1.234567e+33
[LUMINAR_TS_DIAG]                  as_double_ns=1.234567e+33
[LUMINAR_TS_DIAG]                  as_uint32_pair=(lo=3441362965 hi=410895069)
[LUMINAR_TS_DIAG] point[mid]    idx=49152 raw=... as_uint64=1748555700173456789 ...
[LUMINAR_TS_DIAG] point[N-1]    idx=98303 raw=... as_uint64=1748555700223456789 ...
[LUMINAR_TS_DIAG] deltas (mid - first / last - first):
[LUMINAR_TS_DIAG]   as_uint64_ns: mid-first=500000000 last-first=1000000000
[LUMINAR_TS_DIAG]   as_double_sec: mid-first=0.000000000 last-first=0.000000000
[LUMINAR_TS_DIAG] verdict (heuristic; check raw lines to confirm):
```

```
[LUMINAR_TS_DIAG]    uint64 ns interpretation looks plausible
[LUMINAR_TS_DIAG]    (span 100000000 ns ~= 100.000 ms)
[LUMINAR_TS_DIAG] END
```

Key things to read, in order:

1. **fields table** — confirms *which* field is the timestamp and its declared (datatype, count). Anything other than `UINT8×8`, `FLOAT64×1`, or `UINT32×2` is novel.
2. **per-point raw=...** — the 8 timestamp bytes in little-endian byte order. Compare across `point[0]`, `point[mid]`, `point[N-1]` to see if anything changes between points within a scan. If all three raws are bit-identical, timestamps are collapsed at the scan level.
3. **as_uint64 / as_double_sec / as_uint32_pair** — the same bytes interpreted four ways. Only one of these should land in a physically plausible range (see §5).
4. **deltas** — for a healthy 10 Hz scan, last-first should be roughly 100 ms. Whichever interpretation makes that true is the right one.
5. **verdict** — an automated guess from the deltas. *Always* confirm against the raw bytes before acting on it.

5. Decision table: dump pattern → fix

Each row is a self-contained recipe for the LUMINAR branch of copyPointTimeFromCloud in `gicp_localization/src/localization.cc`.

What the dump shows	Most likely encoding	One-line fix
type=UINT8 count=8, as_uint64 is a giant number that <i>matches</i> header.stamp when divided by 1e9, deltas grow 0→~100 ms.	PTP nanoseconds since epoch, packed as little-endian uint64. Matches the team's pcap reader.	No change needed. Current code is correct.
type=UINT8 count=8 but every point has <i>identical</i> raw bytes; deltas are zero.	Driver is filling per-point time with the scan-level stamp. There is no per-point timing in the message.	Set <code>deskew_=false</code> in YAML, or switch the driver to a per-point mode. The current memcpy is mechanically correct; the data is the problem.
type=FLOAT64 count=1, as_double_sec looks like a small fractional second (0.00–0.10), deltas grow toward ~0.1 s.	IEEE-754 double, seconds <i>since scan start</i> . Common in the Luminar ROS2 wrapper.	In the LUMINAR branch, replace the uint64 memcpy with: <code>double d; std::memcpy(&d, tp, 8); ts_ns = (uint64_t)(d * 1e9);</code> and treat it as offset (see §6).
type=FLOAT64 count=1, as_double_sec looks like a wall-clock epoch (~1.7e9), deltas tiny but positive.	IEEE-754 double, seconds <i>since epoch</i> . Less common but seen on some firmware.	Same fix as above but no offset arithmetic; the result is already an absolute time. Subtract header.stamp before deskewing.
type=UINT32 count=2, as_uint32_pair.lo looks like seconds (~1.7e9), hi looks like 0–1e9 ns.	PTP (sec, nsec) pair, low half seconds, high half nanoseconds.	<code>Read uint32_t s, n; memcpy(&s, tp, 4); memcpy(&n, tp+4, 4); ts_ns = (uint64_t)s*1000000000ull + n;</code>
Timestamp field <i>missing entirely</i> (no t/time/timestamp in the fields list).	Driver was launched in a mode that strips per-point time.	Reconfigure the driver. Deskew will already short-circuit (see <code>has_time_field</code> warning).
Deltas in as_uint64 are negative or wildly large (hundreds of seconds).	Byte order or field-offset mismatch — e.g. the field table shows UINT8 count=8 but at a different offset than findTimeField picks up.	Recheck the field offset in the dump, and rename the timestamp field if needed so findTimeField matches it.

6. Where the fix goes in the code

All branch-by-sensor decoding lives in `copyPointTimeFromCloud` in `gicp_localization/src/localization.cc`. The LUMINAR branch currently does the `uint64` `memcpy`:

```
// Current (works when driver emits PTP ns as UINT8[8]):
case dlio::SensorType::LUMINAR: {
    uint64_t ts_ns = 0;
    std::memcpy(&ts_ns, tp, sizeof(uint64_t));
    std::memcpy(&dst.timestamp, &ts_ns, sizeof(uint64_t));
    return;
}
```

Patch templates by case:

FLOAT64 seconds since scan start

```
case dlio::SensorType::LUMINAR: {
    double dt_sec = 0.0;
    std::memcpy(&dt_sec, tp, sizeof(double));
    const uint64_t header_ns =
        static_cast<uint64_t>(header_stamp.sec) * 1000000000ull
        + header_stamp.nanosec;
    const uint64_t ts_ns =
        header_ns + static_cast<uint64_t>(dt_sec * 1e9);
    std::memcpy(&dst.timestamp, &ts_ns, sizeof(uint64_t));
    return;
}
```

(`copyPointTimeFromCloud` will need access to the cloud header stamp — pass it through as an extra argument from `preprocessPointCloud`.)

UINT32 (sec, nsec) pair

```
case dlio::SensorType::LUMINAR: {
    uint32_t s = 0, n = 0;
    std::memcpy(&s, tp, sizeof(uint32_t));
    std::memcpy(&n, tp + 4, sizeof(uint32_t));
    const uint64_t ts_ns =
        static_cast<uint64_t>(s) * 1000000000ull + n;
    std::memcpy(&dst.timestamp, &ts_ns, sizeof(uint64_t));
    return;
}
```

7. Verifying the fix

After applying the patch, rebuild and re-run with the diagnostic *still enabled* (it's a no-op after first cloud anyway), then look for:

(a) Deskew sanity log — in `integratemu/deskew`, the per-point (`t_max - t_min`) should be ~ 0.1 s for a 10 Hz Luminar scan. If you ever see exactly 0 ns or values > 1 s, the decoder is still wrong.

(b) Pose stability during straight-line driving — compare the pre-fix and post-fix localization/pose tracks against `/gps_na/filtered_odom` over the same segment. Lateral RMS should improve, especially during yaw rates > 10 deg/s.

(c) Rerun the diagnostic on the next boot — the dump should now show the *same* raw bytes the patched decoder expects, deltas in the right range, and the verdict line agreeing with the chosen patch.

8. After you're satisfied

The diagnostic is cheap (one cloud, one fprintf burst) and self-documents the driver's behavior on every boot, so we recommend leaving it in. If you do want to remove it, the call site is labelled with the // One-shot timestamp-field diagnostic comment in preprocessPointCloud and the two helpers are in the anonymous namespace above findTimeField.

9. Quick reference card

Question	Answer
Where does the dump go?	stderr of the GICP node, between [LUMINAR_TS_DIAG] BEGIN and END markers.
When does it fire?	Once per node lifetime, on the very first PointCloud2 received.
How do I disable it?	Delete the std::call_once block in preprocessPointCloud; the helpers above can stay (zero overhead).
Does it slow GICP down?	No. After the first cloud the call_once predicate short-circuits (~1 atomic load per cloud).
Where is the fix applied?	LUMINAR branch of copyPointTimeFromCloud in gicp_localization/src/localization.cc.